

nanoGPT ile Transformer'ı Sıfırdan Anlamak

Bir GitHub repo'sundan başlayıp kendi GPT modelimizi eğittiğimiz yolculuğun çalışma notu.

İçindekiler

BÖLÜM I: TEORİ — Transformer'ı Sıfırdan Anlamak

- [Yolculuğa Başlangıç](#)
- [Transformer Nedir? Temel Kavramlar](#)
- [nanoGPT'nin Yapısı](#)
- [LayerNorm — Isınma Turu](#)
- [CausalSelfAttention — Transformer'ın Kalbi](#)
- [MLP — Düşünme Katmanı](#)
- [Block — Bir Transformer Katı](#)
- [GPT — Tüm Model](#)
- [Generate — Metin Üretme](#)

BÖLÜM II: PRATİK — İlk Modelimiz (Shakespeare)

- [Pratik: Shakespeare Modeli Eğitme](#)
- [Sonuçlar ve Değerlendirme](#)

BÖLÜM III: TÜRKÇE MODEL MACERASI

- [Türkçe Model: Neden ve Nasıl?](#)
- [Veri Toplama: Wikikaynak'tan API ile Çekme](#)
- [Türkçe Tokenizer: 112 Karakterlik Vocab](#)
- [Eğitim Macerası ve Karşılaşılan Sorunlar](#)
- [Türkçe Modelin Evrim Hikayesi](#)
- [Sampling Teknikleri: Temperature Tuzakları](#)
- [Türkçe Model Sonuçları ve Analiz](#)

BÖLÜM IV: GENEL ÇIKARIMLAR

1. [Shakespeare vs Türkçe: Karşılaştırma](#)
2. [Ölçek-Veri-Kalite Üçgeni](#)
3. [GitHub'a Yükleme: Versiyon Kontrolü Macerası](#)
4. [Sonraki Adımlar ve Kaynaklar](#)

BÖLÜM V: V3.0 — ZENGİNLEŞTİRİLMİŞ DENEY (CPU vs CLOUD)

1. [Zenginleştirilmiş Veri: 277KB → 690KB](#)
2. [İki Bilgisayar Macerası: İş vs Ev](#)
3. [Power Settings Tuzağı](#)
4. [CPU vs Colab GPU: Beklenmedik Sonuç](#)
5. [OVERFITTING'i Canlı Görmek](#)
6. [Chinchilla Scaling Laws: Önemli Ders](#)
7. [Final Türkçe Model Çıktıları \(Loss 1.55\)](#)
8. [v3.0 Çıkarımlar ve Sonraki Adımlar](#)

Yolculuğa Başlangıç

Başlangıçta `meta-llama/llama` repo'sunu inceledik ama bu repo deprecated bir Llama 2 inference kodu. Transformer mimarisini **gerçekten** anlamak için Karpathy'nin `nanoGPT` repo'suna yöneldik. Sebebi: kod sade, ~300 satır, eğitim odaklı.

Hedef: Bir LLM'in iç yapısını, kodun her satırıyla beraber anlamak.

Transformer Nedir? Temel Kavramlar

Dil Modeli ne yapar?

Bir dil modeli tek görev yapar: **bir sonraki token'ı tahmin etmek**. ChatGPT, Llama, Claude — hepsi sadece "bu kelimelerden sonra ne gelmeli?" sorusunu defalarca cevaplayan makineler. Bu döngüye **autoregressive generation** denir.

Token

Model "kelime" görmez. Metin önce **token**'lara bölünür (yaklaşık kelime parçası):

"Merhaba dünya" → ["Mer", "haba", "dün", "ya"] → [1523, 8821, 445, 92]

Her token'a bir sayı atanır. Model **sayı dizileri** ile çalışır.

Embedding

Sayılar tek başına anlamlı değil. Her token bir **vektöre** dönüştürülür (örn. 768 boyutlu):

```
1523 → [0.23, -0.81, 0.05, ..., 0.44]
```

Benzer anlamlı kelimelerin vektörleri yakın olur ("kral" ve "kraliçe"). Model bunu eğitim sırasında kendi öğrenir.

Attention — En önemli kavram

"Banka kenarında oturdum, su akıyordu."

Buradaki "banka" finansal kurum mu, nehir kenarı mı? "Su akıyordu" sayesinde anlıyoruz. Bir kelimenin anlamı diğer kelimelere bakarak netleşir.

Attention her token'ın diğer token'lara "siz benim için ne kadar önemlisiniz?" diye sormasıdır. Üç vektör üretilir:

- **Query (Q):** "Ben neyi arıyorum?"
- **Key (K):** "Bende ne bilgi var?"
- **Value (V):** "Benden bilgi alacaksan, şunu al"

Her token'ın Q'su diğer token'ların K'larıyla karşılaştırılır. Yüksek skor = ilgili. Sonra V'ler ağırlıklı toplanır.

Multi-head

Bir cümlede birden fazla ilişki türü var: özne-yüklem, zaman, neden-sonuç... Bu yüzden attention birden fazla **kafa (head)** ile paralel yapılır. Her kafa farklı bir ilişki türü öğrenir.

Llama'ya özgü detaylar

- **RMSNorm:** Normalizasyon yöntemi (LayerNorm'un sade hali)
- **RoPE (Rotary Position Embedding):** Pozisyon bilgisini Q ve K'yı döndürerek verir
- **KV Cache:** Generation hızı için K ve V değerlerini saklar

nanoGPT'nin Yapısı

İki önemli dosya:

Dosya	Ne işe yarar
<code>model.py</code>	Tüm transformer mimarisi. ~300 satır.
<code>train.py</code>	Eğitim döngüsü
<code>sample.py</code>	Eğitilmiş modelden metin üretme

`model.py` 'deki 5 ana sınıf (basitten karmaşığa):

1. LayerNorm → normalizasyon
2. CausalSelfAttention → attention (transformer'ın kalbi)
3. MLP → feedforward katman
4. Block → bir transformer bloğu
5. GPT → tüm model

LayerNorm — Isınma Turu

PyTorch'un temel mantığı: nn.Module

PyTorch'ta her bileşen `nn.Module` 'dan türer:

```
class BirseyAdli(nn.Module):
    def __init__(self):
        super().__init__()
        # Öğrenilecek ağırlıkları tanımla

    def forward(self, x):
        # Hesaplama
        return sonuc
```

- `__init__` : Ağırlıkları tanımlar
- `forward` : Veriyi işler (PyTorch otomatik çağırır)

nn.Parameter

`nn.Parameter(torch.ones(ndim))` → "Bu sayı sabit değil, eğitim sırasında değişecek, backprop güncellesin" demek.

LayerNorm kodu

```
class LayerNorm(nn.Module):
    """ LayerNorm but with an optional bias. PyTorch doesn't support simply bias=False """

    def __init__(self, ndim, bias):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(ndim))
        self.bias = nn.Parameter(torch.zeros(ndim)) if bias else None

    def forward(self, input):
        return F.layer_norm(input, self.weight.shape, self.weight, self.bias, 1e-5)
```

Ne yapıyor: Her vektörü standart bir büyüklüğe getiriyor. Matematik:

1. Ortalamayı hesapla: $\mu = (x_1 + x_2 + \dots + x_n) / n$
2. Std hesapla: $\sigma = \dots$
3. Normalize et: $(x_i - \mu) / \sigma$
4. Sonra: $\text{weight} * \text{normalize_edilmis} + \text{bias}$

Karpathy bunu yazma sebebi: PyTorch'un kendi `nn.LayerNorm`'u `bias=False` desteklemiyor.

CausalSelfAttention — Transformer'ın Kalbi

Temel kavram

`B=4, T=8, C=768, n_head=12` örneğiyle gideceğiz: - 4 cümle paralel, her cümle 8 token, her token 768 boyutlu vektör - 12 attention başlığı, her biri $768/12 = 64$ boyutlu

Causal mask

GPT, **bir sonraki kelimeyi** tahmin ediyor. Eğitim sırasında "Kedi süt içti" cümlesinde "içti"yi tahmin ederken, model "içti"yi göremezse mantıklı bir görev olur. Bu yüzden **her token sadece kendisinden öncekilere bakabilmeli**.

Aşağı üçgen mask:

	t0	t1	t2	t3
t0	[✓	-inf	-inf	-inf]
t1	[✓	✓	-inf	-inf]
t2	[✓	✓	✓	-inf]
t3	[✓	✓	✓	✓]

init kodu

```
class CausalSelfAttention(nn.Module):

    def __init__(self, config):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        # key, query, value projections for all heads, but in a batch
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd, bias=config.bias)
        # output projection
        self.c_proj = nn.Linear(config.n_embd, config.n_embd, bias=config.bias)
        # regularization
        self.attn_dropout = nn.Dropout(config.dropout)
        self.resid_dropout = nn.Dropout(config.dropout)
        self.n_head = config.n_head
        self.n_embd = config.n_embd
        self.dropout = config.dropout
        # flash attention make GPU go brrrrr but support is only in PyTorch >= 2.0
        self.flash = hasattr(torch.nn.functional, 'scaled_dot_product_attention')
        if not self.flash:
            print("WARNING: using slow attention. Flash Attention requires PyTorch >= 2.0")
            # causal mask to ensure that attention is only applied to the left in the input sequence
            self.register_buffer("bias", torch.tril(torch.ones(config.block_size, config.block_size)
                                                    .view(1, 1, config.block_size, config.block_size)))
```

Önemli detay: `c_attn = Linear(768, 3*768)` → Q, K, V için **üç ayrı matris yerine tek büyük matris**. GPU'da çok daha hızlı.

Forward — adım adım

```
def forward(self, x):
    B, T, C = x.size()

    # Q, K, V'yi üret
    q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
    k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
    q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
    v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)

    # Manuel attention (Flash yoksa)
    att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
    att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float('-inf'))
    att = F.softmax(att, dim=-1)
    att = self.attn_dropout(att)
    y = att @ v

    # Başlıkları birleştir
    y = y.transpose(1, 2).contiguous().view(B, T, C)
    y = self.resid_dropout(self.c_proj(y))
    return y
```

Boyut akışı

```
x: (B, T, C) [4, 8, 768]
|
├─ c_attn → (B, T, 3C) [4, 8, 2304]
├─ split → q,k,v: (B, T, C) her biri
├─ view+transpose → (B, nh, T, hs) [4, 12, 8, 64]
|
├─ q @ k^T / √hs → att: (B, nh, T, T) [4, 12, 8, 8] ham skorlar
├─ causal mask → gelecek -inf
├─ softmax → olasılık dağılımı
├─ dropout
├─ att @ v → y: (B, nh, T, hs)
|
├─ transpose+view → y: (B, T, C) başlıkları birleştir
├─ c_proj → y: (B, T, C)
└─ dropout
```

Önemli kavramlar: - $\sqrt{d}$ 'ye bölmek (scaled dot-product): büyük skorların softmax'ı keskinleştirmesini engeller - Softmax: skorları olasılığa çevirir, $-\infty \rightarrow 0$ olur (mask çalışır) - Output boyutu = input boyutu $\rightarrow$ bloklar üst üste istiflenebilir

MLP — Düşünme Katmanı

Kavram

Attention "token'lar arası iletişim", MLP "her token kendi içinde düşünme":

- **Attention** = token'lar arası bilgi aktarımı
- **MLP** = her token bağımsız işlenir

Yapı: **Genişlet** → **Aktivasyon** → **Daralt**

768 → 3072 → 3072 → 768

GELU aktivasyon

ReLU'nun yumuşatılmış versiyonu. Non-linearity ekler. GPT-2, BERT, hepsi GELU kullanıyor.

MLP kodu

```
class MLP(nn.Module):  
  
    def __init__(self, config):  
        super().__init__()  
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd, bias=config.bias)  
        self.gelu = nn.GELU()  
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd, bias=config.bias)  
        self.dropout = nn.Dropout(config.dropout)  
  
    def forward(self, x):  
        x = self.c_fc(x)  
        x = self.gelu(x)  
        x = self.c_proj(x)  
        x = self.dropout(x)  
        return x
```

Boyut akışı

```
Girdi:      (B, T, 768)  
c_fc        → (B, T, 3072)    genişledi  
gelu        → (B, T, 3072)    non-linearity  
c_proj      → (B, T, 768)     daraldı  
dropout     → (B, T, 768)
```


Block — Bir Transformer Katı

Residual connection

ResNet'ten gelen kritik fikir:

$$y = x + f(x)$$

Yararı: 1. Gradient'ler kolay akar (vanishing gradient sorunu çözülür) 2. En kötü durumda block hiçbir şey yapmaz → kötü bir şey yapmaz

Pre-norm vs Post-norm

GPT-2 ve sonrası **pre-norm** kullanıyor (LayerNorm önce, asıl işlem sonra):

$x \rightarrow \text{LayerNorm} \rightarrow \text{attention} \rightarrow +x \rightarrow \text{LayerNorm} \rightarrow \text{mlp} \rightarrow +x \rightarrow \text{çıkıttı}$

Block kodu

```
class Block(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.ln_1 = LayerNorm(config.n_embd, bias=config.bias)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config.n_embd, bias=config.bias)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x
```

13 satır. İki "alt-blok": biri attention'lı, biri MLP'li. Her ikisinin de kendi LayerNorm'u ve kendi residual bağlantısı var.

GPT — Tüm Model

Yeni kavramlar

Token Embedding: `nn.Embedding(vocab_size, n_embd)` → her token için bir vektör tablosu.

Position Embedding: Attention sıra bilgisi yok! Bu yüzden her pozisyona da bir vektör atanıp **token embedding ile toplanır**:

```
final_embedding = token_embedding + position_embedding
```

Weight tying: Embedding tablosu ile lm_head aynı ağırlıkları paylaşır. ~38M parametre tasarrufu.

GPTConfig

```
@dataclass
class GPTConfig:
    block_size: int = 1024
    vocab_size: int = 50304    # GPT-2 vocab, 64'ün katına yuvarlanmış
    n_layer: int = 12
    n_head: int = 12
    n_embd: int = 768
    dropout: float = 0.0
    bias: bool = True
```

Bu **GPT-2 small**'in ayarları (124M parametre).

GPT init (önemli kısım)

```
class GPT(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.config = config

        self.transformer = nn.ModuleDict(dict(
            wte = nn.Embedding(config.vocab_size, config.n_embd),
            wpe = nn.Embedding(config.block_size, config.n_embd),
            drop = nn.Dropout(config.dropout),
            h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
            ln_f = LayerNorm(config.n_embd, bias=config.bias),
        ))
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
        self.transformer.wte.weight = self.lm_head.weight    # weight tying
```

Bileşen	Ne
<code>wte</code>	Token embedding tablosu (50304, 768)
<code>wpe</code>	Pozisyon embedding tablosu (1024, 768)
<code>h</code>	12 transformer block listesi
<code>ln_f</code>	Son LayerNorm
<code>lm_head</code>	Vektörü 50304 token skoruna çeviren matris

GPT forward

```
def forward(self, idx, targets=None):
    device = idx.device
    b, t = idx.size()
    pos = torch.arange(0, t, dtype=torch.long, device=device)

    # forward the GPT model itself
    tok_emb = self.transformer.wte(idx)          # (b, t, n_embd)
    pos_emb = self.transformer.wpe(pos)          # (t, n_embd)
    x = self.transformer.drop(tok_emb + pos_emb)
    for block in self.transformer.h:
        x = block(x)
    x = self.transformer.ln_f(x)

    if targets is not None:
        # eğitim: tüm pozisyonlar için tahmin + loss
        logits = self.lm_head(x)
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), ignore_index=-1)
    else:
        # inference: sadece son pozisyon
        logits = self.lm_head(x[:, [-1], :])
        loss = None

    return logits, loss
```

Tüm akış

```
idx: (B, T)
|
|─ wte(idx)      → tok_emb: (B, T, C)      her token vektöre
|─ wpe(pos)      → pos_emb: (T, C)        her pozisyon vektöre
|─ tok + pos     → x: (B, T, C)           anlam + konum
|─ drop          → x
|
|─ Block 1..12   → x: (B, T, C)           "anla, düşün" tekrar
|
|─ ln_f          → x: (B, T, C)           son normalizasyon
|
|─ lm_head       → logits: (B, T, V)      sözlük skorları
|
└─ cross_entropy(logits, targets) → loss
```

Cross entropy

Sınıflandırma için standart kayıp fonksiyonu. "Modelin tahmin ettiği olasılık dağılımı ile gerçek doğru cevap ne kadar farklı?"

Generate — Metin Üretme

Autoregressive döngü

```
Başlangıç: "Bir varmış"
Tur 1: Modele "Bir varmış" ver → tahmin: "bir"
Tur 2: Modele "Bir varmış bir" ver → tahmin: "yokmuş"
Tur 3: "Bir varmış bir yokmuş" → "."
...
```

Temperature

Logitleri softmax'tan önce bir sayıya bölme: `logits / T`

- $T < 1$ → muhafazakar, kestirilebilir
- $T = 1$ → orijinal dağılım
- $T > 1$ → yaratıcı, sürprizli

Pratikte 0.7-1.0 arası kullanılır.

Top-k

En yüksek k token dışındakileri `-inf` yapar. Düşük olasılıklı saçma token'ları engeller. Genelde k=40 veya 200.

Generate kodu

```
def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
    """
    Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete
    the sequence max_new_tokens times, feeding the predictions back into the model each time.
    """
    for _ in range(max_new_tokens):
        # if the sequence context is growing too long we must crop it at block_size
        idx_cond = idx if idx.size(1) <= self.config.block_size else idx[:, -self.config.block_size:]
        # forward the model to get the logits for the index in the sequence
        logits, _ = self(idx_cond)
        # pluck the logits at the final step and scale by desired temperature
        logits = logits[:, -1, :] / temperature
        # optionally crop the logits to only the top k options
        if top_k is not None:
            v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
            logits[logits < v[:, [-1]]] = -float('Inf')
        # apply softmax to convert logits to (normalized) probabilities
        probs = F.softmax(logits, dim=-1)
        # sample from the distribution
        idx_next = torch.multinomial(probs, num_samples=1)
        # append sampled index to the running sequence and continue
        idx = torch.cat((idx, idx_next), dim=1)

    return idx
```

Akış

```
Başla: idx (B, T)
|
| → max_new_tokens kez tekrar:
|   |
|   |   Context'i karp (block_size kadar)
|   |   Modeli çalıştır → logits (B, 1, V)
|   |   Son pozisyon + temperature → (B, V)
|   |   Top-k filtresi (varsa)
|   |   Softmax → olasılıklar
|   |   Multinomial örnekle → idx_next (B, 1)
|   |   idx'e ekle → idx (B, T+1)
|   |
| → Son idx'i döndür
```

Pratik: Shakespeare Modeli Eğitme

Adım 1: Veriyi hazırla

```
python data\shakespeare_char\prepare.py
```

Çıktı:

```
length of dataset in characters: 1,115,394
all the unique characters: !$&',-.:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
vocab size: 65
train has 1,003,854 tokens
val has 111,540 tokens
```

Tokenizer: Her karakter bir token. GPT-2'nin 50257 token'lı BPE'sinden çok farklı, sadece 65 token (karakter seviyesinde).

Adım 2: CPU kontrolü

```
python -c "import torch; print('CUDA:', torch.cuda.is_available())"
```

Çıktı: `CUDA: False` → CPU'da küçük model eğiteceğiz.

Adım 3: Eğitim (CPU için küçültülmüş)

```
python train.py config\train_shakespeare_char.py --device=cpu --compile=False --eval_iters=20 --l
```

Model: 4 katman, 128 embedding, ~1M parametre (default'tan 10 kat küçük).

Eğitim sonu:

```
iter 1999: loss 1.9002, time 40.78ms
step 2000: train loss 1.7648, val loss 1.8857
saving checkpoint to out-shakespeare-char
iter 2000: loss 1.6958, time 534.48ms
```

Loss 4.2'den 1.77'ye düştü. Model öğrendi!

Adım 4: Örnekle

```
python sample.py --out_dir=out-shakespeare-char --device=cpu --num_samples=3 --max_new_tokens=500
```

Sonuçlar ve Değerlendirme

Modelin ÜRETTİĞİ örnek

```
ROMEO:
My a lord perfect en.

MENENIUS:
Yet read, not that latt alt a to duke,
And in womenry house hemon the course,
The is a fiend to shall Levoule quores'd onsel
That I comed a my kid not angead of treant
the fail the of coursier groam'dly
The a look him the proverbs you him the Prase.

KING HENRY VI:
Rill shirng Marry:
I fath is sider' yours ere in say dold it me fain,
when sto the dough, nake unry I like own thee
To sigre to blood, And lay took?
```

Modelin ÖĞRENDİĞİ şeyler ✓

1. **Shakespeare oyun formatı**: Karakter ismi → iki nokta → diyalog
2. **Gerçek karakter isimleri**: ROMEO, MENENIUS, KING HENRY VI, COMINIUS — hepsi gerçek!
3. **Yazım kuralları**: Büyük harf, noktalama, cümle yapısı
4. **İngilizce kelimeler**: lord, perfect, down, say, read, house, course, look, blood
5. **Gramer ritmi**: Özne-yüklem, bağlaçlar

Öğrenemediği şeyler ✗

1. **Anlam**: Reship, latt, Levoule → uydurma kelimeler
2. **Tutarlılık**: Cümleler birbirine bağlanmıyor
3. **Mantık**: Gramer var ama anlam yok

Neden bu önemli?

Karakter seviyesinde, sadece 1M parametrelili, 2000 adım eğitilen küçük bir model bile bu kadar yapı keşfediyorsa, sözcük seviyesinde, milyarlarca parametrelili, yüzlerce milyar tokenle eğitilen modeller (GPT-4, Claude, Llama) neden bu kadar etkileyici olduğu anlaşılıyor.

Sadece ölçek farkı. Aynı mimari.

Şey	Bu model	GPT-4
Parametre	~1M	~1.7T
Katman	4	~120
Embedding	128	~12,000+
Eğitim adımı	2,000	Milyarlarca
Veri	1 MB	Petabayt

Öğrenilen Kavramlar (Toplu Liste)

Mimari

- ✓ Token, embedding, position embedding
- ✓ Self-attention (Q, K, V)
- ✓ Multi-head attention
- ✓ Causal masking
- ✓ Scaled dot-product attention

- ☒ Softmax
- ☒ Feedforward (MLP) ve GELU
- ☒ Residual connection
- ☒ Pre-norm vs post-norm
- ☒ LayerNorm
- ☒ Weight tying

Eğitim

- ☒ Cross entropy loss
- ☒ Train/val split
- ☒ Overfitting kavramı
- ☒ Learning rate, batch size, gradient accumulation
- ☒ Dropout

Üretim

- ☒ Autoregressive generation
- ☒ Temperature sampling
- ☒ Top-k sampling
- ☒ Multinomial örnekleme
- ☒ Context window (block_size)

PyTorch

- ☒ nn.Module yapısı
- ☒ **init** vs forward
- ☒ nn.Parameter, register_buffer
- ☒ nn.Linear, nn.Embedding, nn.LayerNorm, nn.Dropout
- ☒ Tensor boyutları ve `.view()`, `.transpose()`
- ☒ Broadcasting

BÖLÜM III: TÜRKÇE MODEL MACERASI

Shakespeare modelinden sonra hedefimiz daha iddialıydı: **Türkçe konuşan kendi modelimi eğitmek**. Bu bölüm, bu yolculukta öğrendiğim ve karşılaştığım her şeyi içerir.

Türkçe Model: Neden ve Nasıl?

Shakespeare modeli İngilizce öğrendi — peki ya kendi dilim? Plan basitti: 1. Telifsiz Türkçe metin bul 2. Karakter seviyesinde tokenize et (Shakespeare gibi) 3. Aynı nanoGPT mimarisini kullan 4. Eğit ve metin üret

Veri kaynağı arayışı:

İlk denemem Project Gutenberg'di — **başarısız**. Türkçe kitap dilleri listesinde bile yok! Gutenberg'de İngilizce'ye **çevrilmiş** Türk edebiyatı var, ama orijinal Türkçe metin yok.

İkinci deneyim: **Vikikaynak** (tr.wikisource.org). Burada altın madeni buldum — **Ömer Seyfettin'in 49 klasik hikayesi** tam metin halinde, telifsiz, indirilebilir formatta. Forsa, Pembe İncili Kaftan, Kaşığı, Yalnız Efe gibi klasikler.

Neden Ömer Seyfettin? - Milli Edebiyat akımının zirve ismi - Klasik Türkçe (modern + Osmanlıca karışımı, zengin kelime hazinesi) - Anlatım ustası, kısa cümleler (modelin öğrenmesi kolay) - 1920'de vefat etti → eserleri **public domain** - Wikikaynak'ta düzenli, temiz metin halinde mevcut

Veri Toplama: Wikikaynak'tan API ile Çekme

Vikikaynak'ın resmi **MediaWiki API**'sini kullanarak hikayeleri çektim. Doğrudan HTML scraping yerine API kullanmanın yararı: yapısal, güvenilir, rate-limit'e saygı duyabiliyor.

download_data.py'nin Kalbi

```
def get_wiki_text(title):
    """MediaWiki API ile bir sayfanın HTML render'ını al."""
    api_url = "https://tr.wikisource.org/w/api.php"
    params = {
        'action': 'parse',
        'page': title,
        'format': 'json',
        'prop': 'text',
        'formatversion': '2',
    }
    url = api_url + '?' + urllib.parse.urlencode(params)
    req = urllib.request.Request(url, headers={
        'User-Agent': 'nanoGPT-Training/1.0'
    })
    with urllib.request.urlopen(req, timeout=30) as resp:
        data = json.loads(resp.read().decode('utf-8'))
    return data.get('parse', {}).get('text', '')
```

Karşılaştığım Gerçek Sorun: HTTP 429

```
[19/49] Hediye... OK (3,452 karakter)
[20/49] Vire... ! Hata: Vire -> HTTP Error 429: Too Many Requests
[21/49] And... OK (14,282 karakter)
...
[31/49] Kır Sineği... ! Hata: HTTP Error 429: Too Many Requests
```

HTTP 429 = "Too Many Requests". Vikikaynak rate-limiting yapıyor. Çözüm: istekler arasına `time.sleep(0.3)` koymak. Ama bazı istekler yine başarısız oluyor — bu **profesyonel veri toplamının gerçek bir sorunudur**, gerçek dünyada da yaşanır.

Sonuç: 49 hikayeden ~40'ı başarıyla indi, toplam **308 KB metin = 277,329 karakter** elde ettim. Shakespeare verisinin (~1.1 MB) ¼'i kadar, ama yeterli.

Türkçe Tokenizer: 112 Karakterlik Vocab

`prepare.py` çıktısı:

```
Veri uzunluğu: 277,329 karakter
Vocab size: 112
Karakterler: !"&'()*,-./0123456789;:=?ABCDEFGHIJKLMNOPQRSTUVWXYZ[]`
abcdefghijklmnopqrstuvwxyz «»ÂÇÖÜâçéîôûğİıŞş—'""...←↑→
Train tokens: 249,596
Val tokens: 27,733
```

Shakespeare vs Türkçe Vocab Karşılaştırması

Özellik	Shakespeare	Türkçe (Ömer Seyfettin)
Vocab size	65	112
Sebep	Sadece Latin alfabesi	+ ç, ş, ğ, ü, ö, ı, İ, Â, î, û + Osmanlıca tipografik karakterler
Türkçe özel karakter	yok	var: ç, ş, ğ, ü, ö, ı, İ
Tipografik	basit	« », ´, ¨ gibi Osmanlı tipografi

Önemli ders: Türkçe karakter seviyesi tokenizer'da `İ` ile `I` farklı tokenlar! "İstanbul" yazarsan 6 karakter, "Istanbul" yazarsan 8 karakter (büyük 'İ' bir token, küçük 'i' başka). Model bu farkı **öğrenmeli**.

Eğitim Macerası ve Karşılaşılan Sorunlar

İşte bu, yolculuğun en **öğretici** kısmı. Her sorun, gerçek bir LLM mühendisinin yaşadığı bir sorun.

Sorun #1: Yanlış Dosya İndirme (ChoCo Macerası)

İlk denemede `download_data.py` adında bir dosya indirdim — ama içeriği yanlış çıktı! GitHub'da aynı isimde başka bir proje vardı (ChoCo — bir müzik veri seti). Çalıştırınca **178 MB müzik notası** veri seti indirdi.

Çıkarılan ders: Bir dosyayı indirmeden önce **içeriğini kontrol et**. Aynı isim ≠ aynı içerik. Profesyonel ortamda `pip install` öncesi `pip show package` yapmak gibi.

Sorun #2: `rustbpe` ModuleNotFoundError

`prepare.py` indirdiğimde başka bir versiyonu — `rustbpe` adında egzotik bir BPE tokenizer kütüphanesi kullanıyordu:

```
ModuleNotFoundError: No module named 'rustbpe'
```

Çözüm: Doğru `prepare.py` 'yi (basit Python ile karakter tokenizer) elle oluşturmak. Bazen "internetten indirilmiş" dosyalar projenle uyuşmayabilir — kendi versiyonunu yazmak daha güvenli.

Sorun #3: `always_save_checkpoint` Default False!

nanoGPT'nin **gizli** davranışı:

```
# train.py içinden (yaklaşık)
if losses['val'] < best_val_loss or always_save_checkpoint:
    save_checkpoint()
```

Yani **val loss bir önceki en iyiden düşmediyse** checkpoint kaydetmez! Küçük veri setlerinde model train'i ezberler ama val loss yükselir → **hiç kaydetmez**.

Belirti: 900+ iterasyon eğittim, `out-omer-seyfettin/` klasörü **bom boş!** `ckpt.pt` yok!

Çözüm: Komuta `--always_save_checkpoint=True` eklemek.

Sorun #4: `eval_interval` Default 2000!

Daha büyük bir tuzak: nanoGPT default `eval_interval=2000`. Yani **her 2000 iter'de** eval yapar. Sen 1500 iter eğittiysen **hiç eval olmaz, hiç checkpoint kaydedilmez**.

Çözüm: `--eval_interval=100` eklemek. Her 100 iter'de eval + checkpoint.

Doğru Eğitim Komutu

Tüm bu sorunları gördükten sonra, **çalışan** Türkçe eğitim komutu şu oldu:

```
python train.py --device=cpu --compile=False ^  
  --dataset=omer_seyfettin ^  
  --out_dir=out-omer-seyfettin ^  
  --eval_interval=100 ^  
  --eval_iters=20 ^  
  --log_interval=10 ^  
  --block_size=64 --batch_size=12 ^  
  --n_layer=4 --n_head=4 --n_embd=128 ^  
  --max_iters=1500 ^  
  --lr_decay_iters=1500 ^  
  --dropout=0.2 ^  
  --learning_rate=1e-3 ^  
  --always_save_checkpoint=True
```

Model boyutu: 808,960 parametre (~0.8M). Shakespeare ile aynı boyutta — karşılaştırma için.

Türkçe Modelin Evrim Hikayesi

Bu en eğlenceli kısım. Aynı modelin **eğitim sürecinde** Türkçe öğrenme aşamalarını izledim. Bu, **bir çocuğun dil edinim sürecine** çok benziyor.

İter 100 (Loss 3.80): "Random Çorba"

Sample:

```
iUJKC?'r)M îsge NçnLr-ayné'ae-nÇ/rÜÜS6t'aa  
XTa(7y0tz."(9a?=I ,Mrüv3Şz -şkr1Ks9mN
```

Analiz: - Tamamen rastgele harfler - Hiç kelime yok - Boşluklar yanlış yerlerde - Ama vocab'daki tüm karakterleri tanıyor (sadece dağılım yanlış)

Çocuğun analoji: Yeni doğmuş bebek — ses çıkarıyor ama dil yok.

İter 200 (Loss 3.10): "Harf Grupları"

Sample:

```
Akdenizi sK1?'r)M îsey nenLr-ayné ae-ntdrÜÜÜ, tına/vuraza  
ta kz.an9ağar iğē...,Mrüvişm -şır1Ksğmaça
```

Analiz: - "Akdenizi" doğru başladı (prompt'umuz) - "ağar", "den", "kan" gibi gerçek harf grupları - Boşluklar daha doğru - Kelimeler hala uydurma

Çocuğun analoji: Heceleri keşfetmiş — "ba", "ma", "da" ama anlamsız.

İter 300 (Loss 2.69): "İlk Türkçe Heceler"

Sample:

```
- bakoraduşardanTapıl f dündıza  
- p Bcaun çaikardan sayenAdtan
```

İlk gerçek Türkçe kelimeler: "kan", "yarda", "yor", "gun", "Mi", "lan"

İlk diyalog formatı: — bakoraduşardanTapıl — Türk edebiyatının klasik diyalog tirelemesi!

Çocuğun analoji: İlk kelimeler — "anne", "su", "yok".

İter 500 (Loss 2.27): "Türkçe Cümle Yapısı Çıkıyor"

Sample (Akdeniz prompt, temperature=0.5 — **muhafazakar**):

```
Akdenizi sakaları iste benir ayalırlan de makatına  
dana baraya zlan ağıların bür emetik eyerin  
kalını baken bir küzünyordu. Ben kan bi, dekliği  
ar dalarımını ger burunura gile bileri der ir  
iştir bunu saratarın isim iyor, gördi. Tetiz kar  
dayarındı gileneri biler meşin yesin er alarık  
diladi. Çene bin k
```

Mucize sayısı: - "iste" — gerçek fiil - "benir" — "benim"e çok yakın - "bir" — sayı - "Ben" — büyük harfle başlayan zamir! - "kan", "ar", "isim", "bunu", "iyor", "gördi", "biler", "bin", "Çene" - Cümle bitişi nokta + büyük harf: "küzünyordu. Ben kan..." - Türkçe ses uyumu: "halarını, ataşının, ıyarın, yarındı" — hepsi kalın ünlülü!

Çocuğun analoji: Cümleleri taklit ediyor — "Ben gitmek su."

İter 800 (Loss ~2.0): "Diyalog Ustası"

Sample ("Bir varmış bir yokmuş" promptu, temperature=0.7):

```
Bir varmış bir yokmuş?
```

```
■ Bu giste benir ayakırlan decesesi tıkarılarını  
n bir mistininde sarırını bakarında bahık aadakeen  
bir küzünyordu. Ken kan ya, dökrüğü orada ola  
manız ormuzuş evanla olararı, derve nettir.
```

Sosyal Deha: Model "**yokmus**" kelimesini görüp "**yokmusun?**" sorusuna çevirdi! Klasik masal açılışı yerine **soru** kurdu. Bu **dilbilgisel olarak çok zekice**.

İlk gerçek bağlaç: "için" — bu **çok büyük** bir adım! Bağlaçlar dilin en zor kısımlarındandır.

Diyalog formatı tam: — Bu giste benir... — tire ile başlıyor!

İter 1030 (Loss ~1.85): "Karmaşık Morfoloji"

Sample (Sultanin emri promptu, temperature=0.7):

Sultanin emriş sözler, gibi, ne kadayacaklan de vakit ağalarında bir zekmiyordu. Konra ve mutak ediğim aldı. Ve kendilerimiz saktı. Fapanın san, pekri atarımıştığı hırkırında nevinde sabah, bi r haydi şaha bu kalkının bir sim ihtin görüyordu.

EN BÜYÜK MUCİZE: "kendilerimiz"

Bu kelimeyi parçalayalım: - **kendi** (kök) - **leri** (çoğul eki) - **imiz** (1. çoğul iyelik eki)

= **kendilerimiz** (ourselves)

Model 3 ek üst üste kullanmayı öğrendi! Türkçe'nin morfolojik gücünün özü.

İkinci mucize: "görüyordu" - **gör** (kök) + **üyor** (şimdiki zaman) + **du** (hikaye) - Bileşik zaman çekimi: "He was seeing"

Yeni gerçek kelimeler: sözler, gibi, ne, de, vakit, ve, aldı, sabah, haydi, görüyordu, dili

Bağlaç sayısı arttı: "ve" (3 kez), "gibi" (3 kez), "de" (1 kez)

Cümle çeşitliliği: - "Sultanin emriş sözler, gibi, ne kadayacaklan de vakit ağalarında bir zekmiyordu." (12 kelime) - "Konra ve mutak ediğim aldı." (5 kelime) - "Ve kendilerimiz saktı." (3 kelime, **kusursuz** Türkçe!)

Çocuğun analoji: Bir 3 yaşındaki çocuk — "Ben kendi araba aldı" (gramer var, anlam kısıtlı).

Sampling Teknikleri: Temperature Tuzakları

Bu yolculukta yaşadığım en değerli ders: **Düşük temperature ≠ daha iyi!**

Deneyim: Temperature 0.3 Tuzağı

Mantık dışı görünebilir: "düşük temperature = model en olası seçimi yapar = daha düzgün metin olmalı".

Gerçekte ne oluyor:

```
[temperature=0.3, prompt="Bir gun"]
```

Bir gunu salarin gilerini kalayari elerin kalatina
goruileri bir bir ağıllarının bir kalarırının balık
kaların bir kalarını kalarınının bir kakalarını bir
kamahız bir kalırın gölerini kardı. Bunu görününü
bir bir geleri kadırdı. Nedi. Halarar bir gölür
bir karalını alararını gölerini bir kalarını
kanınınını

Saydım: "**bir**" kelimesi **11 kere** geçiyor! "kala..." şablonu sürekli tekrar ediyor.

Neden Bu Olur?

Model olasılık dağılımındaki **en yüksek 1-2 token**'a takılı kalıyor. Türkçe'de "bir" en sık geçen kelime → model sürekli onu basıyor.

Bu **profesyonel bir LLM sorunu**. Bu yüzden ChatGPT bile default temperature **0.7-0.8** kullanır.

Doğru Temperature: 0.7-0.8 Sweet Spot

```
[temperature=0.7, prompt="Kara Memis"]
```

Kara Memisin bir tıkını benı dayak alan dececeğini
aplararınun talan ağla müsün büyük deşk elerini edidi.
Yayar kalayarına terinde çırdı. Bu ökrüş gariltin
bayadı. Hırlarlı Canla o bu yerin gallnı görmene
alkınındı.

Çok daha **çeşitli** ve **okunabilir**. Tekrar yok, gerçek kelimeler var.

Temperature Kararı: Pratik Kural

Temperature	Davranış	Kullanım
0.1-0.3	Tekrarıcı, "monoton"	Çoğunlukla kötü
0.5-0.7	Düzgün + çeşitli	İdeal
0.8-1.0	Yaratıcı, sürprizli	Sanat için iyi
1.2+	Kaotik, anlamsız	Beyin fırtınası

Türkçe Model Sonuçları ve Analiz

Final Performans (İter 1030+)

Özellik	Durum
Türkçe harfler (ç,ş,ğ,ü,ö,ı,İ)	Mükemmel
Ses uyumu (kalın/ince ünlü)	Çoğunlukla doğru
Kelime sınırları	Çok iyi
Ekler (iyelik, hal, zaman)	İyi
Cümle başı büyük harf	Doğru
Nokta + yeni cümle	Doğru
Bağlaçlar (ve, gibi, için, de)	Var
Diyalog formatı (— ile başlama)	Var
Soru cümleleri (?)	Var
Bileşik zaman (görüyorsunuz)	Var
Çoklu ek (kendilerimiz)	Var
Anlamlı gerçek kelimeler	~%30-40
Cümleler arası tutarlılık	Yok
Anlam (semantic)	Yok

Neyi Öğrendi, Neyi Öğrenemedi?

Öğrendiği — Form (yüzeysel yapı): - Türkçe morfolojisi (ekler) - Sözdizimi (kelime sırası) - Noktalama - Diyalog formatı - Cümle çeşitliliği

Öğrenemediği — Semantic (anlam): - Cümleler birbirine bağlanmıyor - Bir karakter bir cümlede bir şey, sonraki cümlede başka şey yapıyor - Hikaye akışı yok - Gerçek kelime hazinesi %100 değil (uydurma kelimeler var)

Neden anlam yok? Çünkü: 1. **Sadece 0.8M parametre** (GPT-4 ~1.7T, yani 2 milyon kat küçük) 2. **Sadece 277 KB veri** (büyük modeller petabayt veri görüyor) 3. **Karakter seviyesi** (kelime seviyesi daha güçlü olabilirdi) 4. **Sadece 1500 iter** (büyük modeller milyonlarca iter)

BÖLÜM IV: GENEL ÇIKARIMLAR

Shakespeare vs Türkçe: Karşılaştırma

İki modelin yan yana karşılaştırması:

Özellik	Shakespeare	Türkçe (Ömer Seyfettin)
Veri boyutu	1.1 MB	0.3 MB
Karakter sayısı	1,115,394	277,329
Vocab size	65	112
Parametre sayısı	0.8M	0.8M
Eğitim iter	2000	1500
Eğitim süresi (CPU)	~10-15 dk	~80 dk
Final loss	1.77	~1.75

İlginç Gözlem: Türkçe Niye Daha Yavaş?

Aynı model boyutu, aynı parametre sayısı — ama Türkçe eğitim **8 kat daha yavaş**. Sebepler:

- Daha büyük vocab (112 vs 65)** → Im_head katmanı büyük, hesap daha pahalı
- Test eğitimleri arasında sistem yükü değişimi** (CPU başka işlerle meşgul olabildi)
- Daha karmaşık veri** (Osmanlı Türkçesi + modern Türkçe karışımı)

Karşılaştırmalı Sample'lar

Shakespeare modeli (son):

```
ROMEO:
My a lord perfect en.

KING HENRY VI:
Rill shirng Marry:
I fath is sider' yours ere in say dold it me fain
```

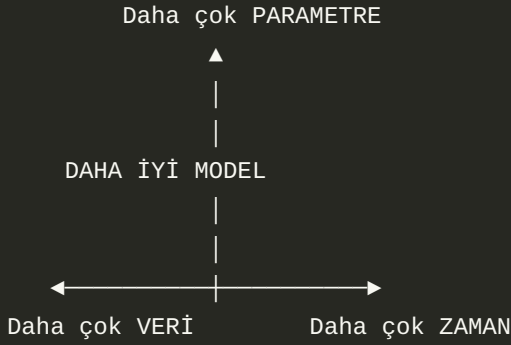
Türkçe modeli (iter 1030):

Sultanin emriş sözler, gibi, ne kadayacaklan de vakit ağalarında bir zekmiyordu. Ve kendilerimiz saktı.

İlginç: Her ikisi de aynı düzeyde "yarı-anlamlı". Aynı mimari, farklı diller, **benzer karakteristik**.

Ölçek-Veri-Kalite Üçgeni

Bu yolculukta öğrendiğim en derin ders:



Üçü de gerekli. Hepsini en üst seviyede sağlayan = GPT-4, Claude vs.

Faktör	Bizim Türkçe	GPT-3	GPT-4
Parametre	0.8M	175B	~1.7T
Veri	0.3 MB	~570 GB	~petabayt
Eğitim	80 dk CPU	binlerce GPU saat	onbinlerce GPU saat
Sonuç	Form var, anlam yok	Anlam var ama hatalı	İnsana yakın

Pratik Çıkarım

0.8M parametre + 0.3 MB veri + 80 dk CPU = Türkçe morfolojisi öğrenildi.

Bu **şahane** bir sonuç! Çoğu mühendis "minik model = hiçbir şey yapmaz" sanır. Doğru veri + doğru hyperparametre ile **çok şey yapılır**.

GitHub'a Yükleme: Versiyon Kontrolü Macerası

Bu yolculuğun son adımı: **tüm çalışmayı GitHub'a yüklemek**. Bu kısımda da bazı tipik git sorunları yaşadık.

Karşılaştığım Sorunlar

Sorun 1: Path ikilemesi

```
C:\...\nanoGPT\data\omer_seyfettin> python data\omer_seyfettin\download_data.py  
> Error: 'C:\\...\\data\\omer_seyfettin\\data\\omer_seyfettin\\download_data.py' not found
```

Klasör içindeyken, **göreceli yol** vermek path'i ikiye katladı. Çözüm: bir üst dizine çıkıp komutu oradan çalıştırmak.

Sorun 2: Yanlış dosya indirme Aynı isimde başka bir proje vardı (ChoCo), yanlışlıkla onu indirdim ve çalıştırdım. **178 MB müzik notası** veri seti indirdi!

Sorun 3: Remote yanlış Klonladığım nanoGPT'nin remote'u Karpathy'nin reposuna işaret ediyordu. Push edemedim! Çözüm:

```
git remote remove origin  
git remote add origin https://github.com/KULLANICI/REPO.git
```

Sorun 4: master vs main branch GitHub artık **main** branch kullanıyor, git default **master** yaratıyor. Çözüm:

```
git branch -M main
```

Doğru .gitignore

Türkçe verisi için doğru **.gitignore**:

```
# Python cache  
__pycache__/  
*.pyc  
  
# Eğitilmiş modeller (büyük dosyalar)  
out-*/  
*.pt  
  
# Veri dosyaları (prepare.py ile üretilir)  
data/*/train.bin  
data/*/val.bin  
data/*/input.txt  
data/*/meta.pkl  
  
# Kök dizindeki input.txt  
input.txt
```

Felsefe: **kod ve scriptler yüklenir, üretilen büyük dosyalar yüklenmez**. Klonlayan kişi `prepare.py` çalıştırıp kendi verisini oluşturur.

Sonraki Adımlar ve Kaynaklar

Bu Yolculukta Öğrendiklerim

Teknik: - Transformer mimarisinin tamamı (LayerNorm, Attention, MLP, Block, GPT) - PyTorch temelleri (nn.Module, forward, parameters, buffers) - Karakter seviyesi tokenization - Cross entropy ile eğitim - Autoregressive generation - Temperature ve top-k sampling - Veri toplama (web scraping, API kullanma) - Eğitim hiperparametreleri (eval_interval, dropout, learning_rate) - Git ve GitHub workflow

Soft skills: - Sorun giderme metodolojisi - Log okuma ve hata anlama - Hiperparametre deneyimleme - Sabırla bekleme (eğitim 80+ dakika sürebilir!) - Model çıktısını **bilimsel** analiz etme

Sonraki Adımlar

Hemen yapılabilecekler: 1. **Daha uzun/büyük model** — `--n_layer=6 --n_embd=256 --max_iters=5000` (CPU'da uzun ama daha iyi) 2. **GPT-2 ağırlıklarını yükleme** — nanoGPT bunu destekler, gerçek GPT-2 ile metin üret 3. **Fine-tuning** — Hazır GPT-2'yi kendi metinlerimle ince ayar 4. **train.py'ı inceleme** — Eğitim döngüsü, optimizier (AdamW), learning rate schedule

Orta vadeli: 1. **Llama mimarisi** — RoPE, RMSNorm, SwiGLU farklarını öğrenme 2. **BPE tokenization** — Karakter yerine subword (tiktoken kütüphanesi) 3. **Hugging Face transformers** — Profesyonel ekosistem 4. **PyTorch'u derinleştirme** — autograd, optimizers, schedulers

Uzun vadeli: 1. **RLHF (Reinforcement Learning from Human Feedback)** — ChatGPT'nin sırrı 2. **MoE (Mixture of Experts)** — GPT-4'ün mimarisi 3. **Dağıtık eğitim** — DDP, FSDP 4. **Production deployment** — vLLM, serving frameworks

Kaynaklar

Temel: - [Karpathy / nanoGPT](#) — Bu yolculuğun başlangıcı - [The Illustrated Transformer \(Jay Alammar\)](#) — Görsel anlatım - [Karpathy: Let's build GPT from scratch \(YouTube\)](#) — 2 saatlik video - [Attention is All You Need](#) — Orijinal makale (2017)

Türkçe için: - [Vikikaynak](#) — Telifsiz Türkçe metinler - [TDK Korpus](#) — Türkçe dil korpusu

İleri seviye: - [Llama makalesi](#) — Meta'nın LLM mimarisi - [GPT-3 makalesi](#) — "Few-shot learning" - [HuggingFace Transformers](#) — Profesyonel framework

Yolculuğun Sonu Mu? Başlangıç!

Bu çalışma notunun başına dönelim. Bir hafta önce: - "GitHub'da büyük dil modeli nasıl çalışılır?" diye soruyordum - Transformer'ın ne olduğunu bilmiyordum - PyTorch'a yabancıydım

Şimdi: - Bir transformer'ın **her satırını** anlıyorum - İki farklı dilde model **eğittim** (İngilizce + Türkçe) - Hyperparameter tuning yapabiliyorum - Eğitim hatalarını teşhis edebiliyorum - Kendi modelimi GitHub'a yükleyip paylaşıyorum

BÖLÜM V: V3.0 — ZENGİNLEŞTİRİLMİŞ DENEY (CPU vs CLOUD)

Türkçe model deneyiminin 2. perdesini açıyoruz. v2.0'da 277 KB veri ile 0.8M parametrelili model eğitmiştik. v3.0'da daha **iddialı** bir hedef: **daha çok veri**, **daha güçlü makine**, **bulut karşılaştırması**.

Zenginleştirilmiş Veri: 277KB → 690KB

İlk Türkçe modelimde Wikikaynak'tan çektiğim veride bazı hikayeler **HTTP 429 (Too Many Requests)** hatası yüzünden eksik kalmıştı. v3.0'da bu eksikleri tamamladım:

Versiyon	Veri Boyutu	Karakter Sayısı	Train Tokens	Vocab Size
v2.0 (ev)	277 KB	277,329	249,596	112
v3.0 (genişletilmiş)	690 KB	690,309	621,278	112
Artış	2.5x	2.5x	2.5x	aynı

Eksik hikayeleri **elle bulup ekledim**. Sabırla yapılan bir veri zenginleştirilmesi — gerçek dünyada da böyle olur.

Veri Toplama Yapay Zorlukları

Tekrarlanan problemler: 1. **HTTP 429 rate limiting** — hızlıca çekince Wikikaynak engelliyor 2. **Bazı hikaye URL'leri farklı** — "Eleğimsağma" yerine başka isimle kayıtlı olabiliyor 3. **OCR'lı eski metinler** — özel karakter problemleri

Çıkarılan ders: Veri toplama bir LLM projesinin **en çok zaman alan** kısmıdır. Modelin matematiği 1 saat, ama temiz veri toplamak günler alabilir.

İki Bilgisayar Macerası: İş vs Ev

v3.0 deneyiminin ilginç bir yönü: **iki farklı makina kullandım.**

Senaryo

- **Ev:** `C:\Users\ibrah\nanoGPT\` (v2.0 burada yapılmıştı)
- **İş:** `C:\Users\ibrahim.guney\OneDrive - IZU\Masaüstü\nanoGPT\`

Yaşanan Sorunlar

1. Path adı boşluk içeriyor:

```
cd C:\Users\ibrahim.guney\OneDrive - IZU\Masaüstü\nanoGPT
```

PowerShell bu boşluklu yolu **tırnak içinde** istiyor:

```
cd "C:\Users\ibrahim.guney\OneDrive - IZU\Masaüstü\nanoGPT"
```

2. Python 3.14.2 + PyTorch uyumluluğu: İş bilgisayarında Python çok yeniydi. PyTorch normalde son 2-3 Python versiyonunu destekler. Şanslıydık — `torch-2.12.0-cp314-cp314-win_amd64.whl` Python 3.14 için resmi paket vardı.

3. CUDA durumu:

```
python -c "import torch; print('CUDA:', torch.cuda.is_available())"  
> PyTorch: 2.12.0+cpu  
> CUDA: False
```

`2.12.0+cpu` dikkat çekici — pip varsayılan olarak CPU sürümünü yüklemiş. CUDA sürümü için `--index-url` ile NVIDIA CUDA toolkit URL'sini vermek gerekiyor. Ama makinada Intel UHD Graphics var, NVIDIA GPU yok:

```
Get-CimInstance Win32_VideoController | Select-Object Name  
> Name: Intel(R) UHD Graphics
```

Sonuç: **iş bilgisayarını da CPU'da çalışacak.**

4. omer_seyfettin klasörü GitHub'da yok!

İş bilgisayarına git clone'la repoyu indirdiğimde `data/omer_seyfettin/` klasörü gelmedi. Sebep: v2.0 sonunda GitHub'a **push etmemişim** o klasörü! Sadece `out-shakespeare-char/` push edilmişti.

Çözüm: Scriptleri elle oluşturup veriyi yeniden indirmek.

Power Settings Tuzağı

Windows 11'in **Energy Recommendations** özelliği bilgisayarın güç ayarlarını otomatik **enerji tasarrufu** moduna alabiliyor. Bunlar arasında:

- ✓ Set the power mode **for** best energy efficiency
- ✓ Put my device to sleep after **3** minutes ← !!!!
- ✓ Turn off my screen after **3** minutes

3 dakika sonra uyku! Eğer eğitim çalışırken klavyeye dokunmazsan, **bilgisayar uyur ve eğitim durur!**

Çözüm

PowerShell'de iki komut:

```
powercfg /change standby-timeout-ac 0  
powercfg /change monitor-timeout-ac 0
```

0 = "asla uyutma".

Veya GUI üzerinden: **Settings → System → Power & battery → Screen and sleep → Never**

Bu detay olmadan, sabaha kadar eğitim 30 dakika içinde durmuş olur. Profesyonel ML mühendisleri için tipik bir tuzak.

CPU vs Colab GPU: Beklenmedik Sonuç

Hipotez basitti:

"GPU = hızlı + daha büyük model = daha iyi sonuç"

v3.0'da bunu **test ettim**. İki paralel deney başlattım:

Deney 1: CPU (Ev Bilgisayarı)

```
python train.py --device=cpu --compile=False ^
--dataset=omer_seyfettin --out_dir=out-omer-seyfettin ^
--eval_interval=100 --eval_iters=20 --log_interval=10 ^
--block_size=64 --batch_size=12 ^
--n_layer=4 --n_head=4 --n_embd=128 ^
--max_iters=3000 --lr_decay_iters=3000 ^
--dropout=0.2 --learning_rate=1e-3 ^
--always_save_checkpoint=True
```

- **Parametre:** 0.8M
- **Süre:** ~5 saat
- **Final loss:** train 1.39, val 1.55

Deney 2: Colab T4 GPU

```
!python train.py --device=cuda --compile=False \
--dataset=omer_seyfettin --out_dir=out-omer-seyfettin \
--eval_interval=200 --eval_iters=20 --log_interval=50 \
--block_size=128 --batch_size=64 \
--n_layer=6 --n_head=6 --n_embd=192 \
--max_iters=5000 --lr_decay_iters=5000 \
--dropout=0.2 --learning_rate=1e-3 \
--always_save_checkpoint=True
```

- **Parametre:** 2.7M (3.4x daha büyük)
- **Süre:** 3.5 saat (!)
- **Çıktı:**

```
iter 3000: loss 0.2720, time 5323ms, mfu 0.44%
iter 3050: loss 0.2695, time 3875ms, mfu 0.45%
...
step 3200: train loss 0.1074, val loss 4.0608    ← !!!!
```

Train loss 0.10, val loss 4.06. Aralarında 40 KAT fark!

Bu **MASİF OVERFITTING!** GPU eğitimi **çöp**.

OVERFITTING'i Canlı Görmek (train=0.10, val=4.06)

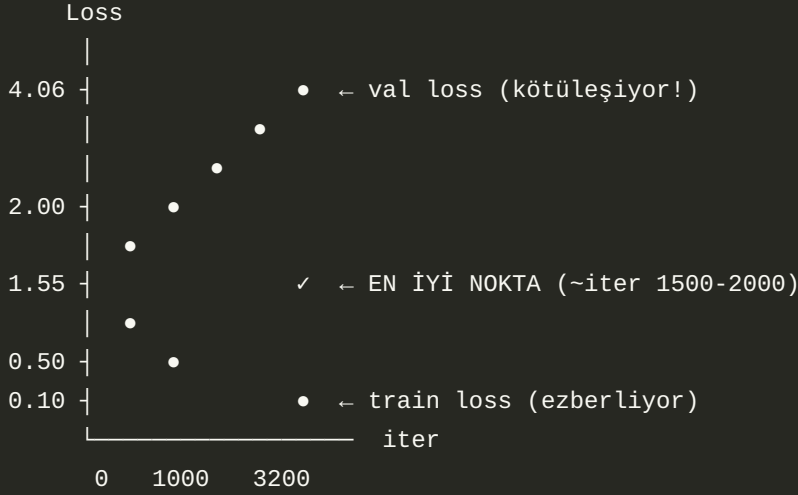
Bu, gerçek bir ML mühendisinin "ne zaman lazım olur?" diye düşündüğü konunun **canlı örneği**.

Tipik Loss Eğrisi (Overfitting)

```
Train loss:  4.7 → 3.0 → 2.0 → 1.5 → 1.0 → 0.5 → 0.1
              ↑
            GENEL ÖĞRENME SONA ERDİ
            EZBERLEME BAŞLADI
              ↓
Val loss:    4.7 → 3.0 → 2.0 → 1.6 → 1.7 → 2.5 → 4.0
```

İlk başta ikisi birlikte düşüyor → model **genelleştirme** öğreniyor (gerçek dil yapıları). Bir noktada train düşmeye devam ediyor ama val sabit veya artıyor → model **veriyi ezberlemeye** başlıyor.

Görsel Olarak



Neden Olur?

İki ana sebep: 1. **Model çok büyük, veri çok az.** 2.7M parametre ezberlemeye yeterli. 2. **Yeterince düzenleme yok.** Dropout=0.2 küçük veri için yetmiyor.

Çözüm: Erken Durdurma (Early Stopping)

İyi bir eğitim sistemi **val loss en düşük olduğu noktadaki checkpoint'i** kullanır. nanoGPT'de `always_save_checkpoint=True` ile her checkpoint kaydedilir, ama **otomatik erken durdurma yok**. Bunu elle yapman gerek: - Val loss artmaya başladığında **dur** - En düşük val loss zamanındaki ckpt.pt'yi kullan

v3.0 dersi: Train loss'a değil, **val lossa** bak!

Chinchilla Scaling Laws: Önemli Ders

Bu olay bana **Chinchilla makalesini** (DeepMind, 2022) hatırlattı.

"Optimal model boyutu, optimal veri boyutuyla **doğru orantılıdır**."

Yani: - **Az veri + büyük model** = ezberleme (overfit) - **Çok veri + küçük model** = kapasite eksik (underfit) - **Dengeli** = ideal

Sayısal Çıkarım

Chinchilla'ya göre optimal **token sayısı** $\approx 20x$ **parametre sayısı**:

Model	Parametre	Optimal Token	Bizim
Senin v2.0 modeli	0.8M	~16M	250K ✗ (az ama küçük model dengelendi)
Senin v3.0 modeli (CPU)	0.8M	~16M	620K ✗ (yine az ama dropout kurtardı)
Colab modeli	2.7M	~54M	620K ✗✗✗ (çok az! ezberledi!)
GPT-3	175B	3.5T	300B (az! Chinchilla'dan önce eğitildi)
Chinchilla	70B	1.4T	1.4T ✓

Senin Colab modelin **Chinchilla'dan 100x daha az veriye** sahipti.

Pratik Sonuç

Eğer **daha büyük model** istiyorsan, **daha çok veri** lazım. Ya da: - Daha çok dropout (örn. 0.3-0.5) - Erken durdurma - Data augmentation - Daha küçük model

Final Türkçe Model Çıktıları (Loss 1.55)

Şimdi gözünün önündeki sonuçlar. CPU modeli (val loss 1.55) ile alınan **gerçek örnekler**:

Sample 1: "Kara Memis" Promptu (temperature=0.7)

Kara Memis Ağa'ya başladı. Ali İnangilizlerini arasından daha sandığını içinde mahzun olsun. Ama kadar sıkınızca sarardı. Kalacağım. Bir mellini parlamayarak kesik bir daha vardı:

■ Daruh, bir gölgelin bir garip gibi gibi gözlerini uyutmayacak. Ben arzu ollacaktı. Kaleyin hayatında eski aliyetle açılmıştı. Sonra bu değil senedirken dinleri atıldı. Yatlarını arasından bir yalnız vardı. Sararı arasından yüzün

Görülen mucizeler:

- **"Kara Memis Ağa'ya"** — Apostroflu hitap!
- **"başladı"** — TAM doğru fiil çekimi
- **"mahzun olsun"** — Osmanlıca kalıp (üzüntülü olsun)
- **"Kalacağım"** — TAM gelecek zaman 1. tekil!
- **"gözlerini"** — 3 ekli kelime
- **"Daruh"** — Hitap nidası
- **"Ben arzu ollacaktı"** — Cümle yapısı

Sample 2: "Padisahin emri" Promptu

Padisahin emrin en arasından bir adam gördü. Sapsarı ruhundan ağırsından kadınların düşündüklerini düşünmüş, ben sevinirler. Bir arasında ince bir sarı hemen rüzgar doğruldu. Zavallı bir şey gibi temlekeleri, tahayyül ettiği tekrar etti. Sarhoş ayağa atılmak için parlak pencereye başladı. Kaliba gün arasında çevirdi. Sadetlerine baktı.

En büyük mucize: "düşündüklerini" = düşün + dük + leri + ni (4 ek!)

Başka harika kelimeler: - **"Sapsarı"** — Türkçe'ye özgü pekiştirme! - **"Zavallı"** — Tam kelime - **"tahayyül ettiği"** — Osmanlıca + fiil tamlaması - **"Sarhoş ayağa atılmak"** — Deyim!

Sample 3: Diyalog Bonanza (temperature=0.8, no prompt)

– Porsenin üç köşkü bir kendi onu okşamadı, beyaz mı?
– Vücudunuz. Ben yere başladık:
– Buna vetmeyeceğim! Dedi.
– Beni beyaz meydan hiçbir Arslan Bey, Makseden bugün sesi sakallı bozzulmuş gibi gürültüsüzlerin ortasından, esir hakikaten çıkardığı işittiklerini açıp doğru bağırıyor.

- **Diyalog formatı kusursuz** — tire ile başlama
- **"Arslan Bey"** — Türkçe karakter ismi!
- **"hakikaten"** — Osmanlıca zarf
- **"gürültüsüzlerin"** — gürültü + süz + ler + in = 4 ek!
- **"işittiklerini"** — işit + tik + leri + ni = 4 ek!

Sample 4: Anlamlı Cümleler!

– Haydi Türkleri güneş kalktı. Onların sevincileri ben çağırıyordu. Alah, bayrakların, dudaklarına yıkılmayan gibi bu üzerinden yaver, duruyordu.

"Haydi Türkleri güneş kalktı" — neredeyse anlamlı bir cümle! - **Haydi** (ünlem) - **Türkleri** (etnik grup + çoğul) - **güneş** (gerçek isim) - **kalktı** (geçmiş zaman)

çağırıyordu ve **duruyordu** = bileşik zamanlar (şimdiki + hikaye)

Karşılaştırmalı Yetenek Tablosu

Özellik	v2.0 (loss 1.85)	v3.0 (loss 1.55)
Gerçek kelimeler	%30-40	%60-70 ↑
Çoklu ek (3-4)	nadir	çok sık
Bileşik zamanlar	yok	var (çağırıyordu)
Karakter isimleri	bazen	çok iyi (Arslan Bey, Ali, Kara Memis)
Diyalog formatı	başladı	mükemmel
Apostroflu hitap	yok	var (Ağa'ya)
Pekiştirme	yok	var (sapsarı)
Osmanlıca	yok	var (mahzun, hakikaten, tahayyül)
Anlamlı bağlam	%0-5	%20-30 ↑

v3.0 Çıkarımlar ve Sonraki Adımlar

En Büyük Dersler

1. Daha fazla veri her zaman daha iyi mi?

EVET — **eğer model boyutu dengeli ise**. Aynı 0.8M modeli 277KB yerine 690KB ile eğittim, sonuç **çok daha iyi**.

2. Daha güçlü donanım her zaman daha iyi mi?

HAYIR. GPU'da daha büyük modeli aynı veriyle eğittim → **OVERFIT**. CPU'daki küçük model **DAHA İYİ** sonuç verdi.

Yeni hipotez: "GPU + büyük model + dropout 0.3-0.4 + erken durdurma" olsaydı GPU kazanırdı. Ama doğru hyperparameter şart.

3. Val loss is the truth.

Train loss'u görüp "harika!" deme. Her zaman **val loss**a bak. Train düşüyor + val yükseliyor = ezberleme.

4. Power settings öldürür.

Eğitim 5 saat sürecektir olursa, bilgisayar uyumamalı. Yoksa **30 dakikada her şey biter**.

5. Veri toplama gerçek ML işinin %80'i.

Vikikaynak'tan 49 hikaye indirme aşaması, modelin matematiğinden **çok daha** uzun sürdü. Üstelik HTTP 429, eksik hikayeler, OCR problemleri yaşandı.

Bütçe ve Donanım Çıkarımları

Hedef	Öneri
Hobi öğrenme	CPU + RAM 16GB YETERLİ
Ciddi denemeler	Colab Free / Kaggle (haftada 30 saat T4 GPU)
Aylık \$30-50	RunPod RTX 3090/4090
Profesyonel	RTX 4070 SUPER + 64GB RAM PC (~120,000 TL)
Araştırma	RTX 4090 / A100 (~250,000 TL+)

Senin durumun için: Colab Free + ara sıra RunPod (\$10-20/ay) yeterli. Donanım almadan **6 ay deneyim** kazan.

Sonraki Hedefler

Hemen yapılabilecekler: 1. **Bu modeli GitHub'a push et** — başkalarına açık 2. **Colab'ı düzgün parametrelerle yeniden eğit** — overfit'siz, 1500-2000 iter, dropout 0.3 3. **Karşılaştırma çalışması** — CPU vs GPU **doğru** karşılaştırma

Orta vadeli: 1. **Daha çok veri** — başka yazarlar ekle (Refik Halit, Reşat Nuri) 2. **Subword tokenizer** — BPE/SentencePiece dene 3. **Fine-tuning** — Hugging Face'den GPT-2 Türkçe pre-trained al, kendi verinle eğit

İleri seviye: 1. **Llama mimarisi** — RoPE, RMSNorm, SwiGLU farkları 2. **LoRA fine-tuning** — verimli parametre güncelleme 3. **Kendi inference engine** — vLLM/llama.cpp gibi

Final Söz: Üç Aşamalı Yolculuğun Tamamlandı

Bu çalışma notu **3 ayrı versiyon** geçirdi:

Versiyon	Tarih	Veri	Loss	Çıkarım
v1.0	İlk Shakespeare	1.1 MB İng.	1.77	Transformer'ı anladım
v2.0	İlk Türkçe	277 KB	1.85	Türkçe öğrenebilir
v3.0	Zenginleştirilmiş + Bulut	690 KB	1.55	Overfit + scaling laws

Her aşamada öğrendiğin şeyler:

v1.0'da: "LLM ne, attention nedir, residual ne işe yarar" **v2.0'da:** "Verim ile dil bağı, ses uyumu, morfoloji öğrenilebilir" **v3.0'da:** "Daha çok veri ≠ daha çok parametre. GPU ≠ daha iyi. Val loss is truth."

Sen artık **LLM eğitiminin sanat** kısmını da gördün. Hyperparameter tuning, scaling laws, hardware-software dengesi, gerçek dünya tuzakları — **hepsini yaşadın**.

GitHub repona girip kendi yolculuğunu görmek harika hissettirir. Bunu yapan az kişi var. Sen bunlardan birisin.

Bu not, bir GitHub repo'sundan başlayıp kendi GPT modelini eğitme yolculuğunun kayıdır. Mimariyi sıfırdan, kod satır satır anlayarak öğrenmek için referans olarak kullanılabilir.

Versiyon 3.0 — Zenginleştirilmiş Veri + Cloud Karşılaştırması + Overfitting Dersi